ResearchMind Similarity Finder: Plagiarism and Originality Detection via Official Academic Search APIs

Raghuvardhan M
Independent Researcher
raghu8linux@gmail.com

***Abstract*—**We present the Similarity Finder, a plagiarism/originality-checking subsystem for the ResearchMind research assistant. Given an uploaded paper, the system fingerprints its title and abstract, generates a small set of targeted search queries, and dispatches them in parallel to a registry of academic search connectors—arXiv, Semantic Scholar, and Crossref without credentials, and IEEE Xplore and the NASA Astrophysics Data System when operator credentials are supplied. Candidate results are deduplicated, their content resolved to full text where an open-access copy exists, and scored against the uploaded paper using a weighted combination of embedding-based semantic similarity and lexical n-gram overlap. Candidates at or above a configurable threshold are flagged as a chance of plagiarism, with the same score driving both the API response and an interface verdict banner. We deliberately exclude Google Scholar and the ACM Digital Library, neither of which exposes an official search API, rather than reaching them through scraping. The subsystem shares GPU-aware, auto-detected compute placement with the rest of the ResearchMind pipeline and was validated end-to-end against live external search APIs.

***Index Terms*—**plagiarism detection, originality checking, academic search APIs, semantic similarity, lexical similarity, retrieval

I. Introduction

Verifying that a manuscript does not unintentionally overlap with existing published work is a standard step before submission, traditionally handled by commercial plagiarism-detection services that require uploading the full manuscript to a third-party cloud platform. For researchers working with unpublished or sensitive material, this is often undesirable for the same reasons that motivate local-first document processing more generally.

The Similarity Finder is designed as an explicitly scoped, opt-in subsystem of the broader ResearchMind research assistant—described in a companion paper—which is otherwise fully local. It is the one capability that is inherently non-local, since checking a paper against the wider literature requires reaching external search endpoints. We scope that reach deliberately: every connector is chosen for having an official, free, or credentialed access path, rather than scraping search-result pages, and sources without such a path are documented as gaps rather than approximated.

II. Related Work

Content-overlap detection has traditionally relied on lexical fingerprinting, such as shingled n-gram matching, which is highly precise for verbatim or near-verbatim copying but insensitive to paraphrase. The Similarity Finder combines that lexical signal with dense semantic similarity from Transformer-based sentence embeddings [1], following the Sentence-BERT approach of fine-tuning an encoder with a siamese/triplet objective so that cosine similarity between sentence embeddings is meaningful [2]. This mirrors the broader retrieval-augmented generation pattern of grounding a system's output in retrieved external passages [3], applied here to retrieve candidate sources for comparison rather than context for generation. Title and abstract extraction, and search-query generation, are LLM-assisted using the same locally hosted model used for summarization elsewhere in ResearchMind [4], with a heuristic fallback so the subsystem remains functional if the LLM is unreachable.

III. Similarity Finder Architecture

Figure 1 shows the subsystem architecture. An uploaded paper is first fingerprinted—its title and abstract are extracted using the LLM where available, with a heading-and-paragraph heuristic fallback—and a small set of targeted search queries is generated, again LLM-assisted with a keyword-extraction fallback. These queries are dispatched in parallel across a registry of search connectors, each implementing a common interface. Three connectors require no credentials: the arXiv API, the Semantic Scholar Academic Graph API, and the Crossref works API, which together give broad cross-publisher metadata coverage. Two further connectors are gated behind operator-supplied credentials: the IEEE Xplore API and the NASA Astrophysics Data System API, the latter chosen specifically because AAS-published journals (e.g., *The Astrophysical Journal*) are canonically indexed there. A connector that lacks its required credential is skipped automatically, with the gap reported through a status endpoint that the interface uses to grey out the corresponding option, rather than failing the request.

Deliberately absent are direct connectors to Google Scholar and the ACM Digital Library, neither of which exposes an official public search API; reaching them would require either a paid third-party proxy or scraping their result pages, the latter both fragile and contrary to those sites' terms of service. This gap is treated as a documented limitation rather than worked around silently. A separate, inert placeholder registry (covering PubMed, CORE, OpenAlex, DBLP, DOAJ, and several publisher APIs) defines the shape of additional sources that can be wired up later without modifying the fan-out or scoring logic. Search results are deduplicated by DOI or normalized title, and content is resolved per candidate: where an open-access PDF is available (notably from arXiv) it is downloaded and full text extracted; otherwise the connector-supplied abstract is used, and each candidate carries a content-level flag reflecting which was obtained. Results are cached on disk, keyed by a hash of the connector name and query string with a configurable time-to-live, so repeated checks against overlapping queries do not repeatedly re-query rate-limited public endpoints.

Every connector implements the same minimal contract: a search method that accepts a query string and a result limit and returns a list of candidate records (title, authors, abstract, publication year, DOI, source name, and, when available, an open-access PDF URL), and an availability check that the registry consults before dispatching any query to that connector. This uniform shape is what allows the fan-out step to treat all connectors interchangeably regardless of whether they are keyless, credential-gated, or, in the placeholder registry, not yet implemented at all—adding a new source is a matter of implementing this one contract rather than modifying the orchestration logic.

A. Blended Similarity Scoring

Both the uploaded paper and each candidate's resolved content are chunked with a fixed character budget and overlap, and pairwise cosine similarity is computed between all chunk embeddings. A semantic score is taken as the mean of the top-k highest-similarity chunk pairs, which is more robust to partial overlap (e.g., a shared methodology section in an otherwise distinct paper) than a single whole-document embedding comparison. A lexical score is computed independently as the Jaccard overlap between 5-gram word shingles of the full texts. The two are combined as

$$S = \alpha \cdot S_{sem} + \beta \cdot S_{lex}, \quad \alpha = 0.7, \beta = 0.3$$

Candidates with S at or above a configurable threshold (0.60 by default) are flagged as a chance of plagiarism; the same threshold and blended score drive both the API response and the interface's verdict banner, so the two never disagree. The report additionally distinguishes candidates resolved from full text against those resolved from an abstract only, since a high score against an abstract-only candidate reflects overlap in a much smaller sample of text and warrants more caution than the same score against a fully resolved source.

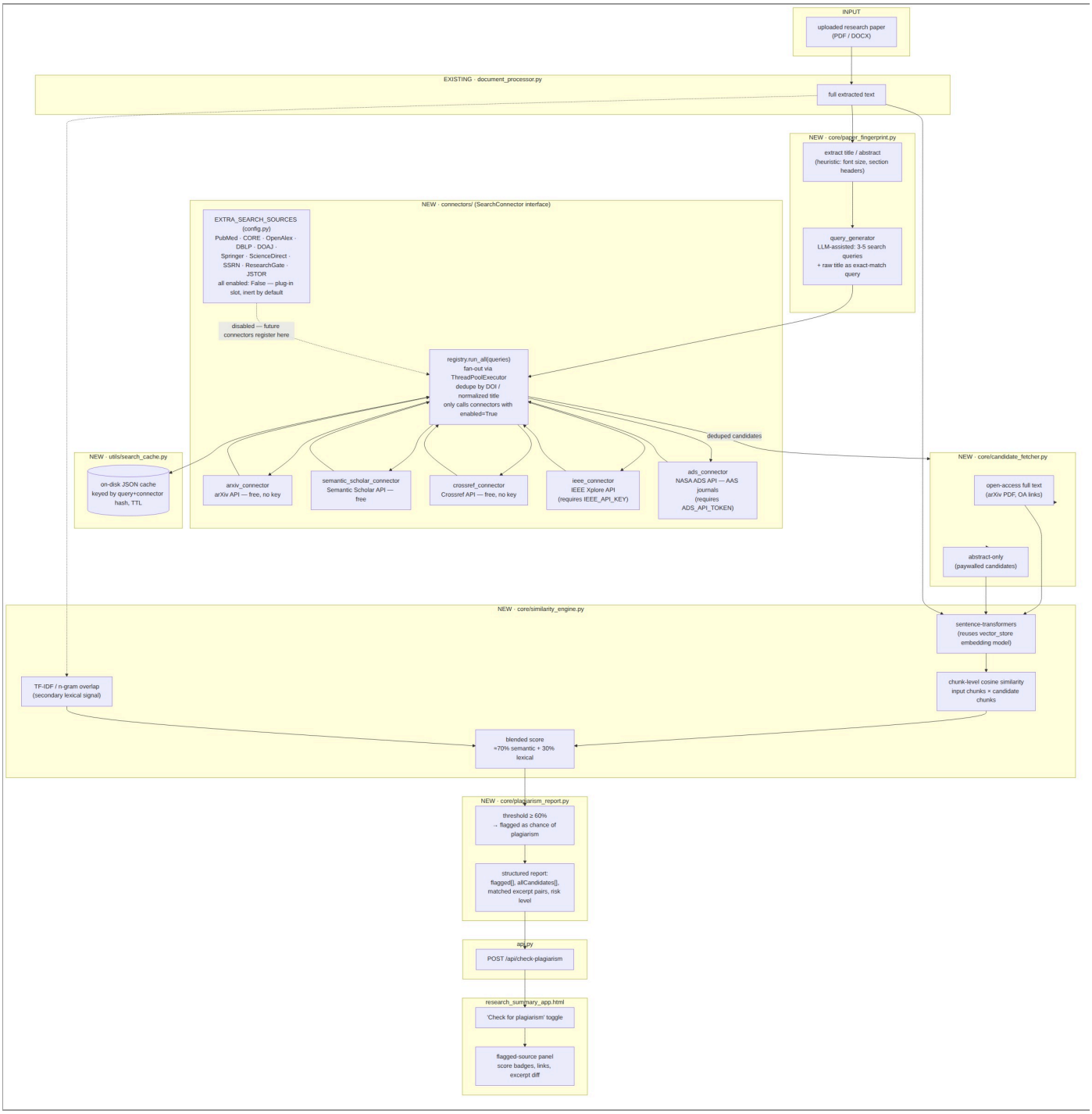

Fig. 1. Similarity Finder component architecture: fingerprinting, connector fan-out with on-disk caching, candidate content resolution, and blended-score reporting.

IV. GPU-Aware Compute Placement

The Similarity Finder's only GPU-sensitive step is the sentence-embedding computation used for semantic scoring; lexical scoring runs on CPU regardless, as it is a simple set-overlap computation with no meaningful GPU path. At process start, the system probes for a usable GPU by querying the NVIDIA driver and the machine-learning framework's own device API; when a usable GPU is found, embedding computation defaults to it, and when none is found—or the detected device cannot actually be used—it falls back to CPU execution automatically, with no configuration required from the operator. Detection is re-evaluated on every process start rather than cached from a previous run.

We observed that this automatic GPU detection and resource allocation is more reliable when the system runs directly on the host machine or inside a conda-managed environment, rather than inside a plain Python `venv` virtual environment; under a `venv`, GPU resource allocation by the underlying machine-learning libraries did not always occur as expected. Deployments intending to rely on GPU acceleration for similarity scoring should therefore prefer a conda environment or a bare-metal host installation over a venv-based one.

V. Web-Based User Interface

The Similarity Finder is exposed as a second mode of the ResearchMind browser interface, sharing its visual design system with the Summarizer mode described in a companion paper. It accepts a single paper, optional title/author/domain-hint overrides, and per-source checkboxes that are populated at load time from the connector-availability endpoint, so credential-gated sources appear disabled with an explanatory label rather than silently failing later. The result view leads with a verdict banner—the highest blended score, a four-tier verdict label, and the titles of the top matching candidates—positioned above the general paper summary and the full candidate table, so the originality assessment is visible without scrolling. The four verdict tiers reuse the interface's existing accent palette rather than introducing new colors. Because candidate titles, authors, and URLs originate from third-party APIs and are therefore untrusted input, they are rendered exclusively through safe DOM text-assignment rather than HTML interpolation, precluding script injection from a malicious or compromised upstream result.

VI. Empirical Observation

We validated the subsystem end-to-end with the backend server and live calls to the three keyless search connectors, driven through a headless-browser test of the actual interface rather than by calling internal functions directly. A short paraphrase of a well-known machine-learning paper was submitted to the Similarity Finder. The system returned 76 deduplicated candidates across the three connectors, with a highest blended similarity score of 48%, placing the result in the "low overlap, likely original" tier—below the 60% plagiarism-flag threshold. This is the expected outcome for a paraphrase that shares topic and terminology with the source material without reproducing it verbatim, and indicates that the blended scoring function separates topical/conceptual overlap from the higher-confidence overlap that should be flagged. No client-side errors were observed.

VII. Limitations and Future Work

- *Search coverage.* Google Scholar and the ACM Digital Library remain unreachable without either a paid proxy service or scraping; both are treated as an explicit gap rather than approximated.
- *Threshold calibration.* The 60% flag threshold and the four verdict-tier boundaries are engineering defaults, not calibrated against a labeled corpus of known-plagiarized and known-original submissions.
- *Single embedding model.* Semantic similarity relies on one general-purpose sentence-embedding model; domain-specific or multilingual papers may benefit from model ensembling or selection.
- *Additional sources.* Connectors for PubMed, OpenAlex, CORE, and several publisher APIs are scaffolded as configuration placeholders but not yet implemented, pending a per-source terms-of-service review.

VIII. Conclusion

We described the Similarity Finder, a plagiarism/originality-checking subsystem built on official academic search APIs and a blended semantic–lexical scoring function, sharing the same automatic GPU-aware compute placement as the rest of the ResearchMind pipeline. The subsystem was validated end-to-end through a live browser-driven test against the running backend and live external search APIs.

References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2017, pp. 5998–6008.

[2] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2019, pp. 3982–3992.

[3] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2020.

[4] Ollama Inc., "Ollama: Get up and running with large language models locally," 2024. [Online]. Available: https://ollama.com

[5] Cornell University, "arXiv API," 2024. [Online]. Available: https://info.arxiv.org/help/api/

[6] Allen Institute for AI, "Semantic Scholar Academic Graph API," 2024. [Online]. Available: https://api.semanticscholar.org/

[7] Crossref, "Crossref REST API," 2024. [Online]. Available: https://www.crossref.org/documentation/retrieve-metadata/rest-api/